options that Rails provides is to not include Active Support libraries at all:

```
config.active_support.bare = true
```

Alternatively, we can direct our logs to the system logs for our machine:

```
config.logger = SyslogLogger.new
```

Find more options at http://guides.rubyonrails.org/configuring.html.

Moving On to Launch and Beyond

Once we've set up our initial deployment, we're ready to finish the development of our application and launch it into production. We'll likely set up additional deployment servers, and the lessons we learn from our first deployment will tell us a lot about how we should structure later deployments. For example, we'll likely find that Rails is one of the slower components of our system: more of the request time will be spent in Rails than in waiting on the database or filesystem. This indicates that the way to scale up is to add machines to split up the Rails load.

However, we might find that the bulk of the time a request takes is in the database. If this is the case, we'll want to look at how to optimize our database activity. Maybe we'll want to change how we access data. Or maybe we'll need to custom-craft some SQL to replace the default Active Record behaviors.

One thing is for sure: every application will require a different set of tweaks over its lifetime. The most important activity is to listen to it over time and discover what needs to be done. Our job isn't done when we launch our application. It's actually just starting.

Although our job is just starting when we first deploy our application to production, we've completed our tour of the Depot application. After we recap what we did in this chapter, let's look back at what we've accomplished in remarkably few lines of code.

What We Just Did